Day 2

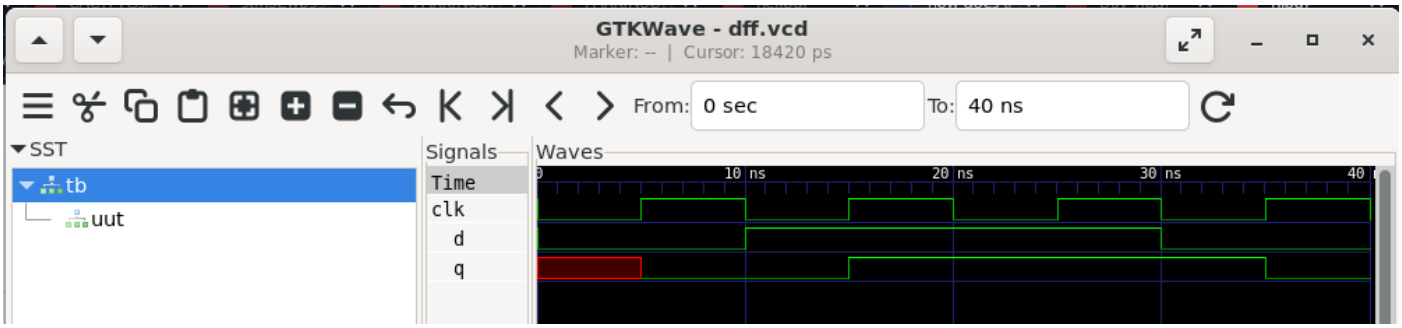
D Flip Flop

Verilog Code:

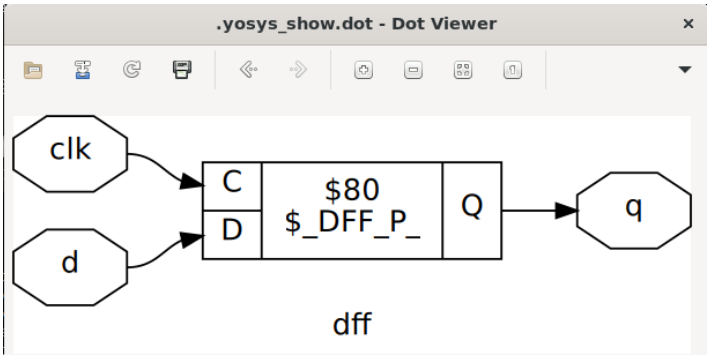
```
module dff(input clk, d,
 output reg q);

always @(posedge clk) begin
 q <= d;
end
endmodule
```

Testbench:

```
`timescale 1ns/1ps
module tb;
reg clk, d;
wire q;
dff uut(.clk(clk), .d(d), .q(q));
initial begin
 $dumpfile("dff.vcd");
 $dumpvars(0, tb);
end
initial begin
clk = 0;
forever #5 clk = ~clk;
end
initial begin
 d = 0; #10;
 d = 1; #10;
 d = 1; #10;
 d = 0; #10;
$finish;
end
endmodule
```



Simulation and Synthesis output:



2.26. Printing statistics.

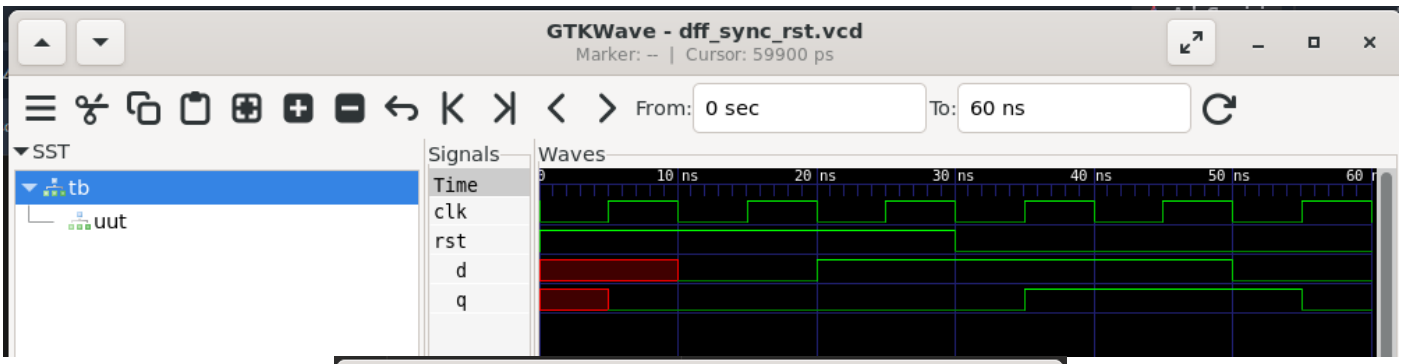
```
=== dff ===

   Number of wires:                 3
   Number of wire bits:             3
   Number of public wires:          3
   Number of public wire bits:      3
   Number of memories:              0
   Number of memory bits:           0
   Number of processes:             0
   Number of cells:                 1
     $_DFF_P_                       1
```

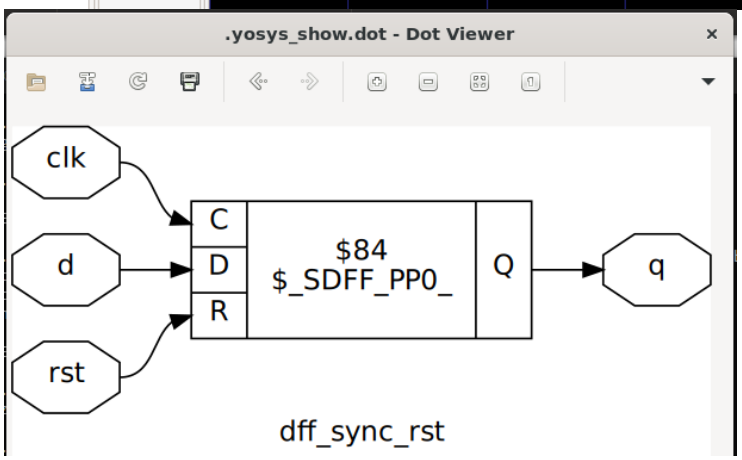
D Flipflop using synchronous reset

```
module dff_sync_rst(input clk, rst, d,
 output reg q);

always @(posedge clk) begin
 if(rst)
   q <= 1'b0;
 else
   q <= d;
end
endmodule
```

Testbench:

```
`timescale 1ns/1ps
module tb;
reg clk, d, rst;
wire q;

dff_sync_rst uut(.clk(clk), .d(d), .q(q), .rst(rst));
initial begin
 $dumpfile("dff_sync_rst.vcd");
 $dumpvars(0, tb);
end

initial begin
clk = 0;
forever #5 clk = ~clk;
end
initial begin
 rst = 1; #10;
 d = 0; #10;
 d = 1; #10;
 rst = 0; #10;
 d = 1; #10;
 d = 0; #10;
$finish;
end
```



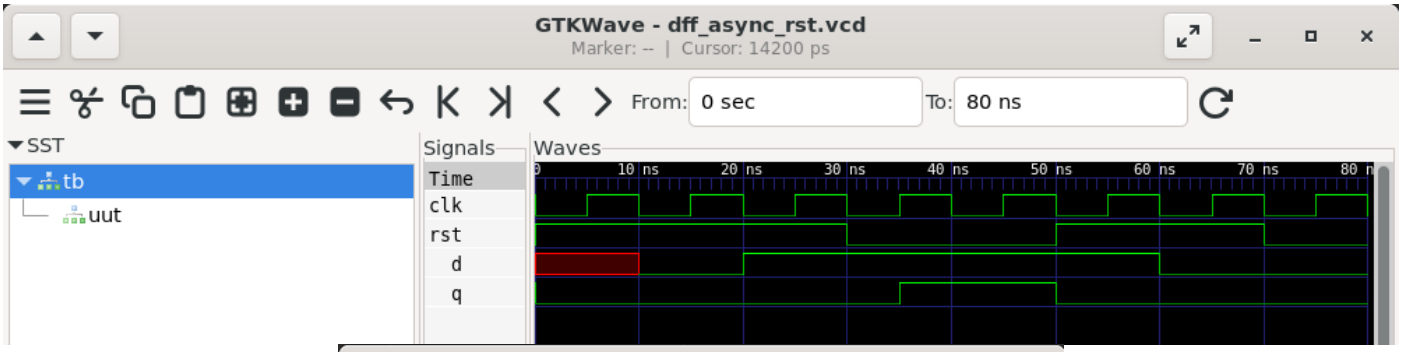


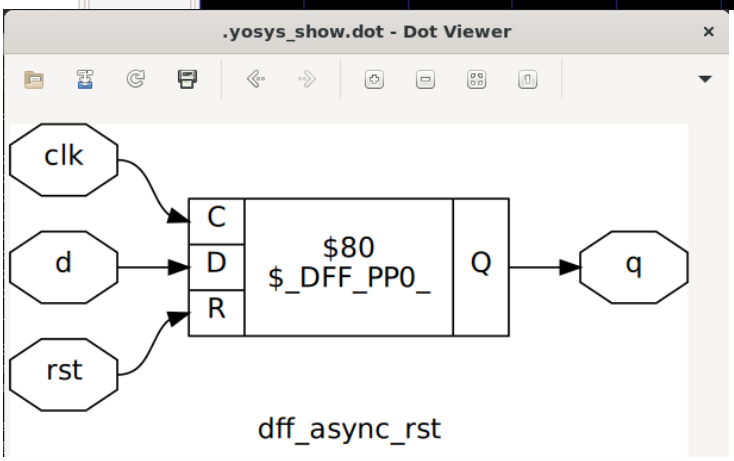
2.26. Printing statistics.

```
=== dff_sync_rst ===

   Number of wires:                 4
   Number of wire bits:             4
   Number of public wires:          4
   Number of public wire bits:      4
   Number of memories:              0
   Number of memory bits:           0
   Number of processes:             0
   Number of cells:                 1
     $_SDFF_PP0_                    1
```

D Flipflop with asynchronous reset

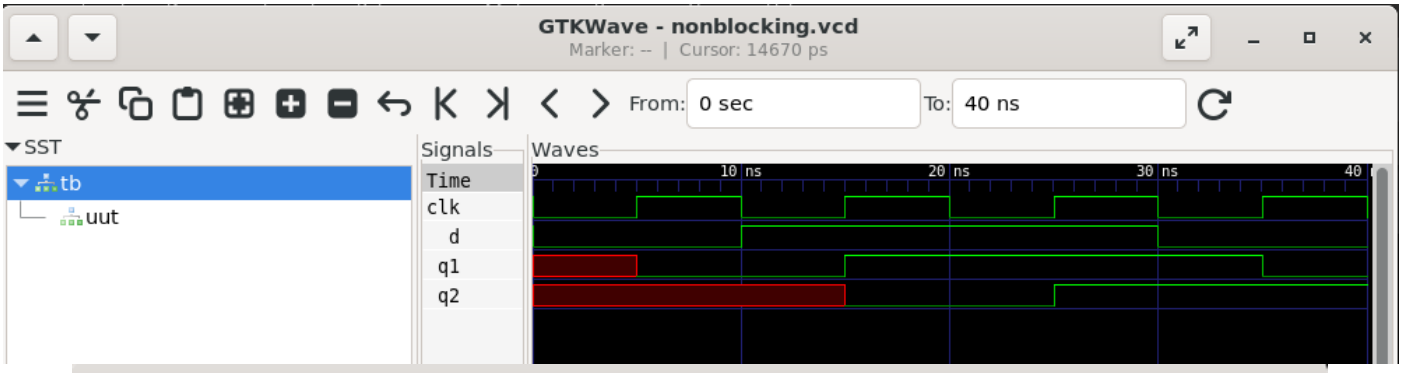
```
module dff_async_rst(input clk, rst, d,
 output reg q);

always @(posedge clk or posedge rst) begin
 if(rst)
   q <= 1'b0;
 else
   q <= d;
end
endmodule
```

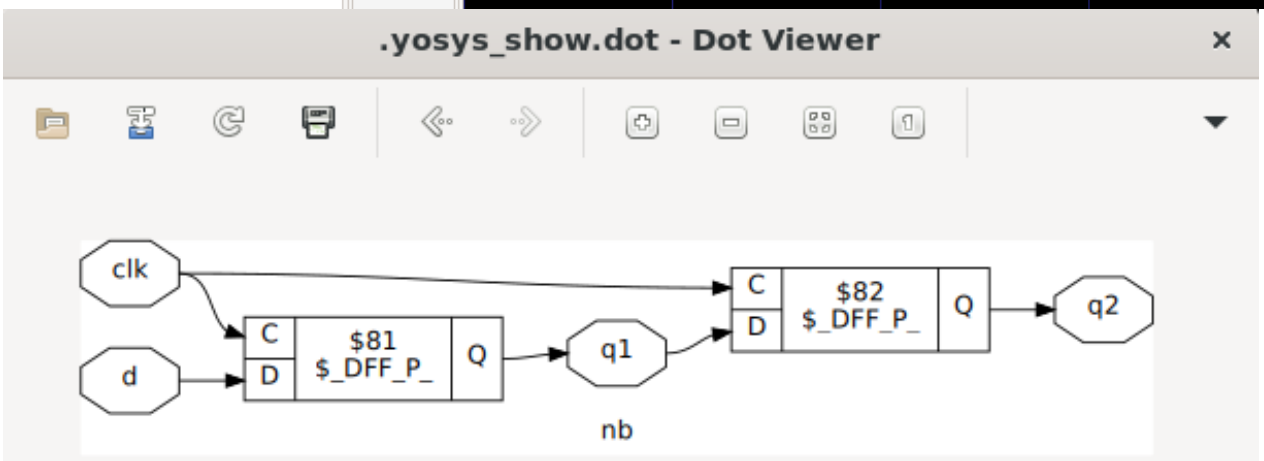




Example using non-blocking assignment:

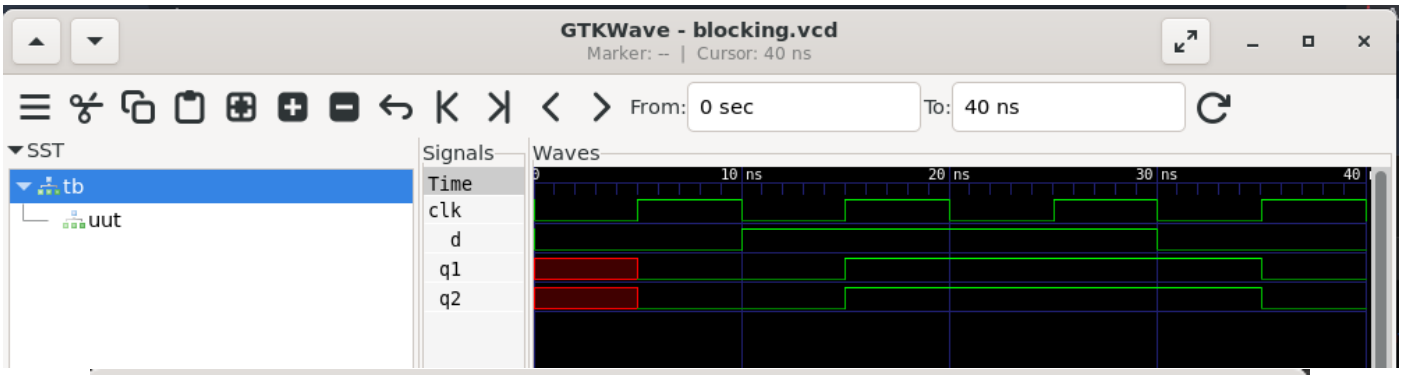
```
module nb(input clk, d, output reg q1, q2);
  always @(posedge clk) begin
    q1 <= d;
    q2 <= q1;
  end
endmodule
```

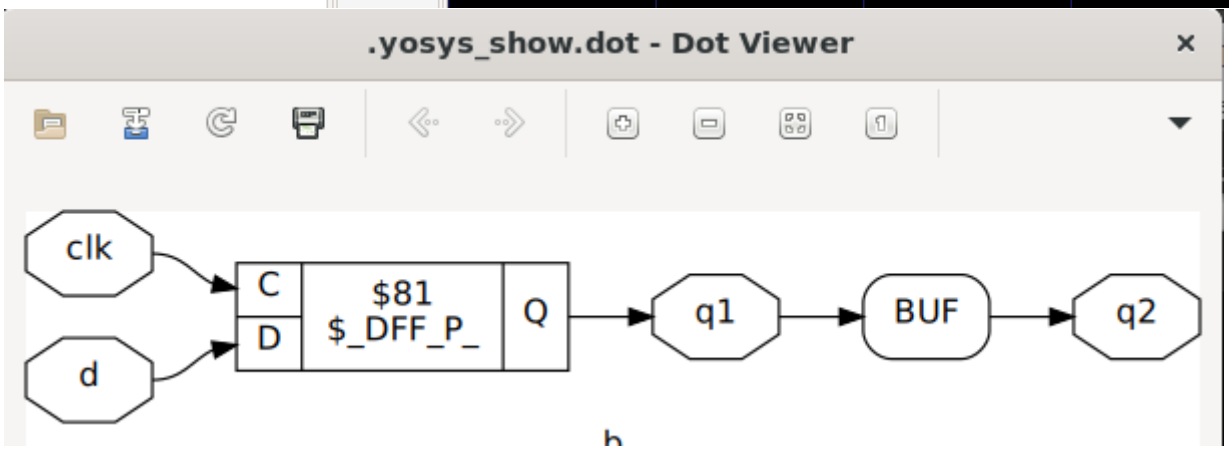




Example using Blocking assignment

```
module b(input clk, d, output reg q1, q2);
  always @(posedge clk) begin
    q1 = d;
    q2 = q1;
  end
endmodule
```





Important Insights:

With blocking assignments, q2 immediately receives the new value of q1 (which is d), creating a wire instead of a register chain. In simulation, this might appear to work, but synthesized hardware will behave differently. The simulator evaluates statements in order, but actual hardware has no such ordering--all flip-flops clock simultaneously.

---- So we should always use non-blocking assignment when dealing with clocked blocks to model the register behaviour correctly

Non-blocking assignments schedule updates for the end of the time-step .This creates a simulation/synthesis mismatch--synthesis tools infer combinational logic (no registers), but simulation behaves as if there are registers.

---- Hence, we should always use blocking assignment during combinational circuits to ensure immediate evaluation.